

Verifica randomizzata del prodotto di matrici

Uno degli usi più comuni degli algoritmi randomizzati è la **verifica di correttezza di un programma** (*program checking*).

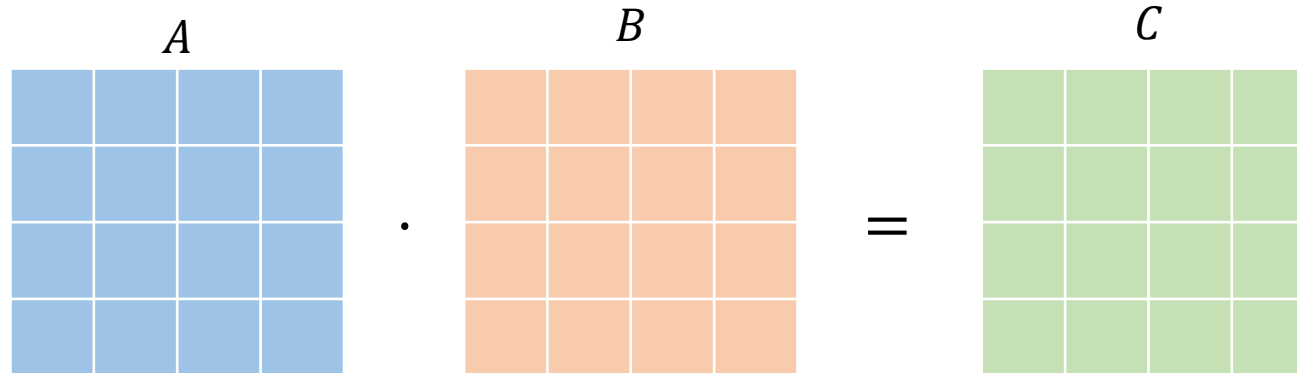
Il caso tipico è quello in cui si dispone di un programma di cui **non ci si fida completamente**.

Si desidera **verificare rapidamente** la correttezza del risultato — in **molto meno tempo** di quanto servirebbe per **ripetere l'intero calcolo da zero**.

Verifica randomizzata del prodotto di matrici

Usando la moltiplicazione standard, per calcolare il prodotto $C = A \cdot B$ di due matrici $n \times n$, A e B occorrono $\Theta(n^3)$ moltiplicazioni.

Ogni elemento di C richiede n moltiplicazioni e ci sono n^2 elementi.



L'algoritmo più efficiente richiede $O(n^{2.371339})$ operazioni (SODA 2025).

Verifica randomizzata del prodotto di matrici

Supponiamo che qualcuno abbia una **versione implementata** dell'algoritmo più efficiente: potremmo **non fidarci del suo codice (perché l'algoritmo è di difficile implementazione)**.

Vorremmo allora **verificare l'esattezza di ogni risultato** che l'implementazione produce. Cioè, ogni volta che si forniscono due matrici A e B e l'algoritmo restituisce C , vogliamo **controllare se davvero $A \cdot B = C$** .

Naturalmente, potremmo calcolare $A \cdot B$ direttamente per verificare, ma ciò richiederebbe ancora tempo $\Theta(n^3)$, **vanificando il vantaggio** dell'algoritmo più veloce.

Algoritmo di Freivalds

Mostreremo ora un **algoritmo Monte Carlo dovuto a Freivalds (1977)** che permette di verificare se $A \cdot B = C$ usando soltanto $\Theta(n^2)$ moltiplicazioni.

L'idea si basa sulla semplice osservazione seguente.

Per **ogni** vettore $\mathbf{r} = (r_1, r_2, \dots, r_n)$, deve valere

$$A \cdot B \cdot \mathbf{r} = C \cdot \mathbf{r}.$$



Proprietà associativa
del prodotto matriciale



$$A \cdot (B \cdot \mathbf{r}) = C \cdot \mathbf{r}.$$

Algoritmo di Freivalds

Mostreremo ora un **algoritmo Monte Carlo dovuto a Freivalds (1977)** che permette di verificare se $A \cdot B = C$ usando soltanto $\Theta(n^2)$ moltiplicazioni.

Algoritmo di Freivalds per la verifica del prodotto di matrici

Input: matrici A, B, C di dimensione $n \times n$, con elementi reali.

1. Scegli un **vettore casuale** $\mathbf{r} = (r_1, r_2, \dots, r_n)$, dove $r_i \in \{0,1\}$.
2. Calcola $B\mathbf{r}$.
3. Calcola $A(B\mathbf{r})$.
4. Calcola $C\mathbf{r}$.
5. Se $A(B\mathbf{r}) \neq C\mathbf{r}$, allora restituisci " $A \cdot B \neq C$ ".
Altrimenti, restituisci " $A \cdot B = C$ ".

Algoritmo di Freivalds

Algoritmo di Freivalds per la verifica del prodotto di matrici

Input: matrici A, B, C di dimensione $n \times n$, con elementi reali.

1. Scegli un **vettore casuale** $\mathbf{r} = (r_1, r_2, \dots, r_n)$, dove $r_i \in \{0,1\}$.
2. Calcola $B\mathbf{r}$.
3. Calcola $A(B\mathbf{r})$.
4. Calcola $C\mathbf{r}$.
5. Se $A(B\mathbf{r}) \neq C\mathbf{r}$, allora restituisci " $A \cdot B \neq C$ ".
Altrimenti, restituisci " $A \cdot B = C$ ".

L'algoritmo usa $\Theta(n^2)$ operazioni: ognuno dei passi 2, 3 e 4 consiste nel moltiplicare una **matrice per un vettore** e quindi usa solo $\Theta(n^2)$ moltiplicazioni.

Algoritmo di Freivalds

Domanda: Se l'algoritmo di Freivalds restituisce " $A \cdot B \neq C$ ", è corretto?

Sì. Se fosse $A \cdot B = C$, allora, per **ogni** vettore $\mathbf{r}$, deve valere $A \cdot B \cdot \mathbf{r} = C \cdot \mathbf{r}$, e quindi $A(B\mathbf{r}) \neq C\mathbf{r}$ non potrebbe mai verificarsi.

Errore unilaterale (*one-sided error*).

L'unico caso in cui l'algoritmo di Freivalds può **sbagliare** è quando restituisce $A \cdot B = C$ quando invece $A \cdot B \neq C$.

Dimostreremo che la **probabilità** che ciò avvenga, cioè la probabilità che l'algoritmo dia una risposta sbagliata, è $\leq \frac{1}{2}$.

Algoritmo di Freivalds

Osservazione: Scegliere $\mathbf{r} = (r_1, r_2, \dots, r_n)$ indipendentemente e uniformemente a caso in $\{0,1\}^n$, **equivale** a scegliere ciascun r_i indipendentemente e uniformemente in $\{0,1\}$.



Scegliendo ciascun r_i indipendentemente e uniformemente in $\{0,1\}$ si ottiene che ognuno dei 2^n possibili vettori di $\{0,1\}^n$ viene generato con probabilità 2^{-n} , cioè con la stessa distribuzione che si ottiene scegliendo $\mathbf{r} = (r_1, r_2, \dots, r_n)$ indipendentemente e uniformemente a caso in $\{0,1\}^n$.

Algoritmo di Freivalds

Osservazione: Scegliere $\mathbf{r} = (r_1, r_2, \dots, r_n)$ indipendentemente e uniformemente a caso in $\{0,1\}^n$, **equivale** a scegliere ciascun r_i indipendentemente e uniformemente in $\{0,1\}$.



Uniformità. Scegliendo $\mathbf{r} = (r_1, r_2, \dots, r_n)$ uniformemente a caso in $\{0,1\}^n$, si ottiene che ciascun r_i assume i valori 0 e 1 con uguale probabilità

$$P\{r_i = 0\} = P\{r_i = 1\} = 2^{n-1}/2^n = \frac{1}{2}.$$

Indipendenza. Per ogni $S \subseteq \{1, 2, \dots, n\}$ e $(a_1, a_2, \dots, a_{|S|}) \in \{0,1\}^{|S|}$

$$P\{\forall i \in S: r_i = a_i\} = \frac{2^{n-|S|}}{2^n} = 2^{-|S|} = \prod_{i \in S} P\{r_i = a_i\}.$$

Algoritmo di Freivalds

Teorema 22.3 (Errore di Freivalds)

Siano A, B, C tre matrici $n \times n$ tali che $A \cdot B \neq C$, e sia $\mathbf{r}$ un vettore scelto uniformemente a caso da $\{0,1\}^n$. Allora

$$P\{A \cdot B \cdot \mathbf{r} = C \cdot \mathbf{r}\} \leq \frac{1}{2}.$$

Dim. Poiché $A \cdot B \neq C$, sappiamo che $D \equiv A B - C \neq \mathbf{0}$, dove $\mathbf{0}$ è la matrice $n \times n$ formata da soli zeri.

Supponiamo ora che il vettore $\mathbf{r}$ sia tale che

$$D\mathbf{r} = A\mathbf{B}\mathbf{r} - C\mathbf{r} = \mathbf{0}.$$

Algoritmo di Freivalds

Supponiamo ora che il vettore $\mathbf{r}$ sia tale che $D\mathbf{r} = A\mathbf{B}\mathbf{r} - C\mathbf{r} = \mathbf{0}$.

$$\begin{bmatrix} d_{11} & d_{12} & d_{13} & \cdots & d_{1n} \\ d_{21} & d_{22} & d_{23} & \cdots & d_{2n} \\ d_{31} & d_{32} & d_{33} & \cdots & d_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ d_{n1} & d_{n2} & d_{n3} & \cdots & d_{nn} \end{bmatrix} \cdot \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_n \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Poiché $D \neq \mathbf{0}$, D deve avere **almeno un elemento non nullo**. Per comodità di notazione, supponiamo che tale elemento sia d_{11} (non si perde di generalità).

Algoritmo di Freivalds

Supponiamo ora che il vettore $\mathbf{r}$ sia tale che $D\mathbf{r} = A\mathbf{B}\mathbf{r} - C\mathbf{r} = \mathbf{0}$.

$$\begin{bmatrix} d_{11} & d_{12} & d_{13} & \cdots & d_{1n} \\ d_{21} & d_{22} & d_{23} & \cdots & d_{2n} \\ d_{31} & d_{32} & d_{33} & \cdots & d_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ d_{n1} & d_{n2} & d_{n3} & \cdots & d_{nn} \end{bmatrix} \cdot \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_n \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Poiché $D\mathbf{r} = \mathbf{0}$, sappiamo che il **prodotto della prima riga** di D per il vettore $\mathbf{r}$ dà 0, cioè

$$\sum_{j=1}^n d_{1j} r_j = 0.$$

Algoritmo di Freivalds

Poiché $D\mathbf{r} = \mathbf{0}$, sappiamo che il **prodotto della prima riga** di D per il vettore $\mathbf{r}$ dà 0, cioè

$$\sum_{j=1}^n d_{1j} r_j = 0.$$

Ma questo implica che

$$d_{11}r_1 + \sum_{j=2}^n d_{1j}r_j = 0 \quad \Rightarrow \quad r_1 = -\frac{\sum_{j=2}^n d_{1j}r_j}{d_{11}}.$$

La probabilità di sbagliare coincide con la probabilità che la scelta casuale di r_1 coincida

con il valore di $-\frac{\sum_{j=2}^n d_{1j}r_j}{d_{11}}$.

Algoritmo di Freivalds

Abbiamo osservato che scegliere $\mathbf{r} = (r_1, r_2, \dots, r_n)$ uniformemente a caso in $\{0,1\}^n$, equivale a scegliere ciascun r_i indipendentemente e uniformemente in $\{0,1\}$.

Le componenti di $\mathbf{r} = (r_1, \dots, r_n)$ sono **scelte in modo indipendente e uniforme** da $\{0,1\}$.

L'ordine in cui le estraiamo **non influisce sulla distribuzione congiunta** del vettore casuale. Quindi possiamo **senza perdita di generalità** immaginare di fissare prima $r_2, \dots, r_n$, e poi considerare r_1 come l'unica variabile ancora libera.

Consideriamo l'istante immediatamente successivo alla scelta di $r_2, \dots, r_n$: a quel punto il valore $-\frac{\sum_{j=2}^n d_{1j} r_j}{d_{11}}$ **è già determinato**. Quanto è probabile che la scelta casuale di r_1 coincida con questo valore?

Algoritmo di Freivalds

$$r_1 = -\frac{\sum_{j=2}^n d_{1j} r_j}{d_{11}}$$

Sia r_1^* il valore del membro destro dell'equazione. Vogliamo determinare $\mathbf{P}(r_1 = r_1^*)$.

- Se $r_1^* \notin \{0,1\}$, allora l'evento $(r_1 = r_1^*)$ è impossibile, quindi la probabilità è 0.
- Se $r_1^* \in \{0,1\}$, allora la probabilità che r_1 coincida con esso è $1/2$.

$$\mathbf{P}(r_1 = r_1^*) = \mathbf{P}(r_1 = r_1^* \cap r_1^* \in \{0,1\}) \leq \mathbf{P}(r_1 \in \{0,1\}) = \frac{1}{2}.$$

Quindi, l'algoritmo sbaglia con probabilità al più $\frac{1}{2}$. Non soddisfacente? OK, ma possiamo ripetere l'esperimento...

Algoritmo di Freivalds iterato

Ripetizione del test (k volte).

Eseguiamo l'algoritmo di Freivalds usando k vettori casuali $\mathbf{r}$.

Se per **almeno un** vettore $\mathbf{r}$ risulta $A(B\mathbf{r}) \neq C\mathbf{r}$, allora **concludiamo** " $A \cdot B \neq C$ "; altrimenti **concludiamo** " $A \cdot B = C$ ".

- Se $A \cdot B = C$, allora **tutte** le k esecuzioni restituiscono "uguale" e la risposta è corretta.
- Se $A \cdot B \neq C$, ogni esecuzione ha **probabilità indipendente** $\leq \frac{1}{2}$ di **scoprire** (fare da *testimone* di) questa disuguaglianza.

Algoritmo di Freivalds iterato

La **probabilità di non scoprire** che $A \cdot B \neq C$ dopo k ripetizioni è $\leq \left(\frac{1}{2}\right)^k$.

Scegliendo, ad esempio, $k = 100$ il **tempo complessivo** è ancora $\Theta(n^{2.37})$:

$$\Theta(n^{2.37} + 100n^2) = \Theta(n^{2.37}),$$

con una probabilità di errore che è al più 2^{-100} : una probabilità astronomicamente bassa.

Verifica randomizzata di polinomi

Supponiamo di avere un programma che dichiari di saper moltiplicare monomi. Ad esempio, il nostro programma potrebbe ricevere in input una sequenza di tre monomi:

$$(x - 3), (x - 5), (x + 7)$$

e produrre in output il polinomio di terzo grado:

$$G(x) = x^3 - x^2 - 41x + 105.$$

Obiettivo: vogliamo un modo per **controllare rapidamente** l'output del programma di moltiplicazione, in **molto meno tempo** del suo tempo di esecuzione (dove per “tempo di esecuzione” contiamo il **numero di moltiplicazioni**).

Verifica randomizzata di polinomi

Definiamo $F(x)$ come il **vero** prodotto dei monomi:

$$F(x) \equiv (x - 3)(x - 5)(x + 7),$$

mentre $G(x)$ è l'**output del programma**.

Il nostro compito è stabilire se $G(x)$ è **uguale** al prodotto $(x - 3)(x - 5)(x + 7)$ **senza dover calcolare** $F(x)$.

Verifica randomizzata di polinomi

Per moltiplicare d monomi la soluzione più semplice richiede $O(d^2)$ moltiplicazioni. Esiste un algoritmo più efficiente che usa $O(d \log^2 d)$ moltiplicazioni.

Presentiamo ora un **algoritmo Monte Carlo** che, con **alta confidenza**, stabilisce se il nostro programma è corretto usando soltanto $\Theta(d)$ moltiplicazioni.

Come prima, partiremo da un **algoritmo semplice** che può commettere errori e poi lo **miglioreremo** per **ridurre la probabilità di errore**.

Verifica randomizzata di polinomi

Algoritmo 22.4 — Verifica semplice per la moltiplicazione di d monomi

Scegli un intero r **uniformemente a caso** tra 1 e $100d$.

Valuta $F(r)$. Valuta $G(r)$.

- Se $F(r) = G(r)$, **dichiara** che il programma è corretto;
- altrimenti, **dichiara** che è errato.

Domanda: Qual è il running time del nostro algoritmo?

Risposta: $\Theta(d)$. Sostituendo x con r , servono solo $O(d)$ moltiplicazioni.

Verifica randomizzata di polinomi

Algoritmo 22.4 — Verifica semplice per la moltiplicazione di d monomi

Scegli un intero r **uniformemente a caso** tra 1 e $100d$.

Valuta $F(r)$. Valuta $G(r)$.

- Se $F(r) = G(r)$, **dichiara** che il programma è corretto;
- altrimenti, **dichiara** che è errato.

Domanda: In quali condizioni sbaglia?

Risposta: Sbaglia **solo** quando essendo $F \neq G$, venga scelto r tale che $F(r) = G(r)$.

Verifica randomizzata di polinomi

Supponiamo che $F(x) \neq G(x)$. Definiamo $H(x) = F(x) - G(x)$.

La probabilità che l'algoritmo dia una risposta sbagliata coincide con la probabilità che un intero r scelto **uniformemente a caso** nell'intervallo $[1, 100d]$ sia tale che

$$H(r) = 0.$$

Questa è la probabilità che r sia una radice del polinomio $H(x)$.

$H(x)$ essendo un polinomio di grado al più d , ha **al più d radici** (dato che $H(x) \neq 0$).

Verifica randomizzata di polinomi

Di conseguenza, ci sono **al più** d valori di r tali che $H(r) = 0$. Cioè, ci sono **al più** d valori di r per cui $F(r)$ e $G(r)$ **coincidono**.

Siccome r viene scelto nell'insieme $\{1, \dots, 100d\}$, la probabilità di estrarre una delle al più d radici di $H(x)$, è al più

$$\frac{d}{100d} = \frac{1}{100}.$$

Verifica randomizzata di polinomi

Pertanto, il nostro *Simple Checker* determina correttamente se il programma è giusto con **probabilità almeno** $99/100$, usando soltanto $\Theta(d)$ operazioni.

Domanda: Supponiamo di voler sapere che il nostro programma è corretto con probabilità $\frac{999999}{1000000}$. Come possiamo modificare il *Simple Checker* per ottenere questa garanzia più elevata?

Risposta: Un'idea è **estrarre r da un intervallo più ampio**. Ad esempio, usare l'intervallo $\{1, \dots, 10^6 d\}$. Tuttavia, ciò **potrebbe non essere praticabile**: aumentare enormemente l'intervallo da cui si estrae r comporta problemi pratici. Ogni moltiplicazione con numeri grandi richiede più tempo. La complessità elementare cresce con la lunghezza in bit dei numeri.

Verifica randomizzata di polinomi

Alternativa più efficiente.

Per ottenere un'affidabilità così alta (0.999999), è **più efficiente ripetere** il test alcune volte con diversi r scelti in un intervallo moderato (ad esempio $1, \dots, 100d$):

$$P(\text{errore con } k \text{ ripetizioni}) \leq \left(\frac{1}{100}\right)^k.$$

Già con $k = 3$ si raggiunge un'affidabilità $\frac{999999}{1000000}$.

Verifica randomizzata di polinomi

Algoritmo 22.5 — Superior Checker

Ripeti il *Simple Checker* k volte, ogni volta scegliendo un valore r **uniformemente a caso** tra gli interi dell'intervallo $[1, 100d]$.

- Se $F(r) = G(r)$ per **tutti** i k valori scelti, **dichiara** che il programma è corretto;
- altrimenti, **dichiara** che il programma è errato.

Osserviamo che il tempo di esecuzione del *Superior Checker* è soltanto $\Theta(kd) = \Theta(d)$ per k costante.

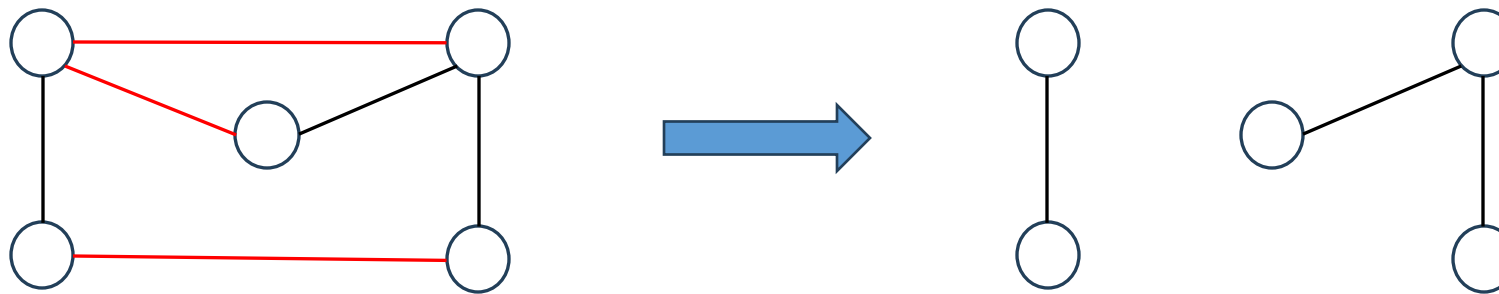
Min-Cut randomizzato

Abbiamo un grafo **non orientato (senza cappi)**

$G = (V, E)$ con $|V| = n$ vertici e $|E| = m$ archi.

Definizione 22.6 — Insieme di taglio (cut-set).

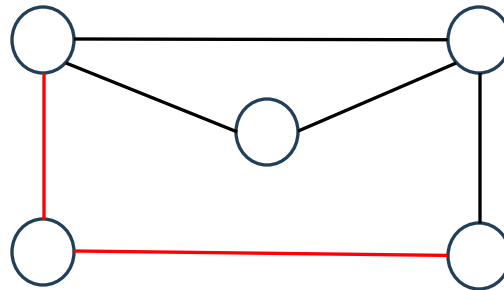
Un **cut-set** di un grafo $G = (V, E)$ è un insieme di archi la cui **rimozione** spezza il grafo in **due o più componenti connesse**.



Min-Cut randomizzato

Definizione 22.7 — Taglio minimo (min-cut).

Un **taglio minimo** è un insieme di taglie di **cardinalità minima**, cioè il più piccolo insieme di archi la cui rimozione disconnette il grafo.



Min-Cut randomizzato

Problema del taglio minimo (Min-Cut Problem): trovare un taglio minimo di un grafo.

Possono esistere **più insiemi di taglio di cardinalità minima**: ci basta trovarne **uno qualunque**.

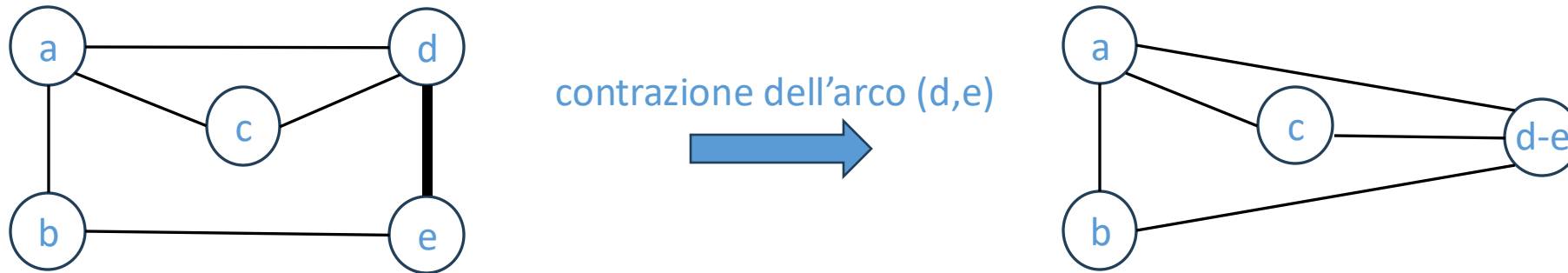
Il problema del *min-cut* ha **numerosi applicazioni**, in particolare nei contesti di **affidabilità dei sistemi e delle reti**.

Ad esempio: qual è il **numero minimo di collegamenti** che possono guastarsi prima che la rete diventi disconnessa?

Min-Cut randomizzato

Presenteremo ora un **algoritmo randomizzato** per trovare un taglio minimo che risulta **più veloce e più semplice** di qualunque algoritmo deterministico noto.

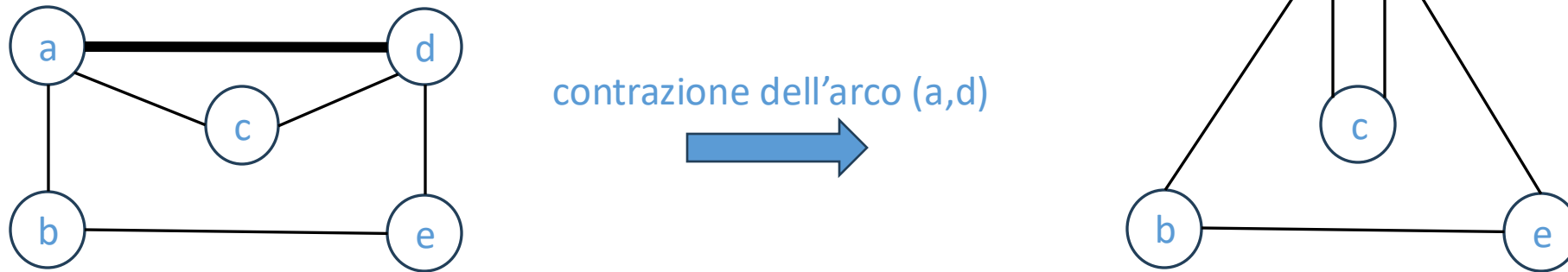
L'idea di base dell'algoritmo è quella di “**contrarre gli archi**” progressivamente, fino a ottenere un insieme di taglio.



Min-Cut randomizzato

Presenteremo ora un **algoritmo randomizzato** per trovare un taglio minimo che risulta **più veloce e più semplice** di qualunque algoritmo deterministico noto.

L'idea di base dell'algoritmo è quella di “**contrarre gli archi**” progressivamente, fino a ottenere un insieme di taglio.



Min-Cut randomizzato

Definizione 22.8 — Contrazione di un arco

La **contrazione** di un arco (v_1, v_2) consiste nel **fondere** i vertici v_1 e v_2 in un **unico vertice** v .

L'arco (v_1, v_2) viene **eliminato**.

Tutti gli archi che avevano come estremità v_1 o v_2 (ma non entrambi) ora avranno come estremità il **nuovo vertice contratto** v .

Si noti che il nuovo grafo può contenere **archi paralleli**, ma **nessun cappio**.

Algoritmo per il taglio minimo

Algoritmo 22.9 — Algoritmo randomizzato per il taglio minimo

1. Dato un grafo G , **ripeti** finché restano solo due vertici u e v :
 - i. Scegli **un arco a caso** tra tutti quelli attualmente presenti nel grafo;
 - ii. **Contrai** quell'arco (fondendo i due vertici estremi).
2. **Restituisci** l'insieme degli archi che connettono i due vertici rimasti u e v .

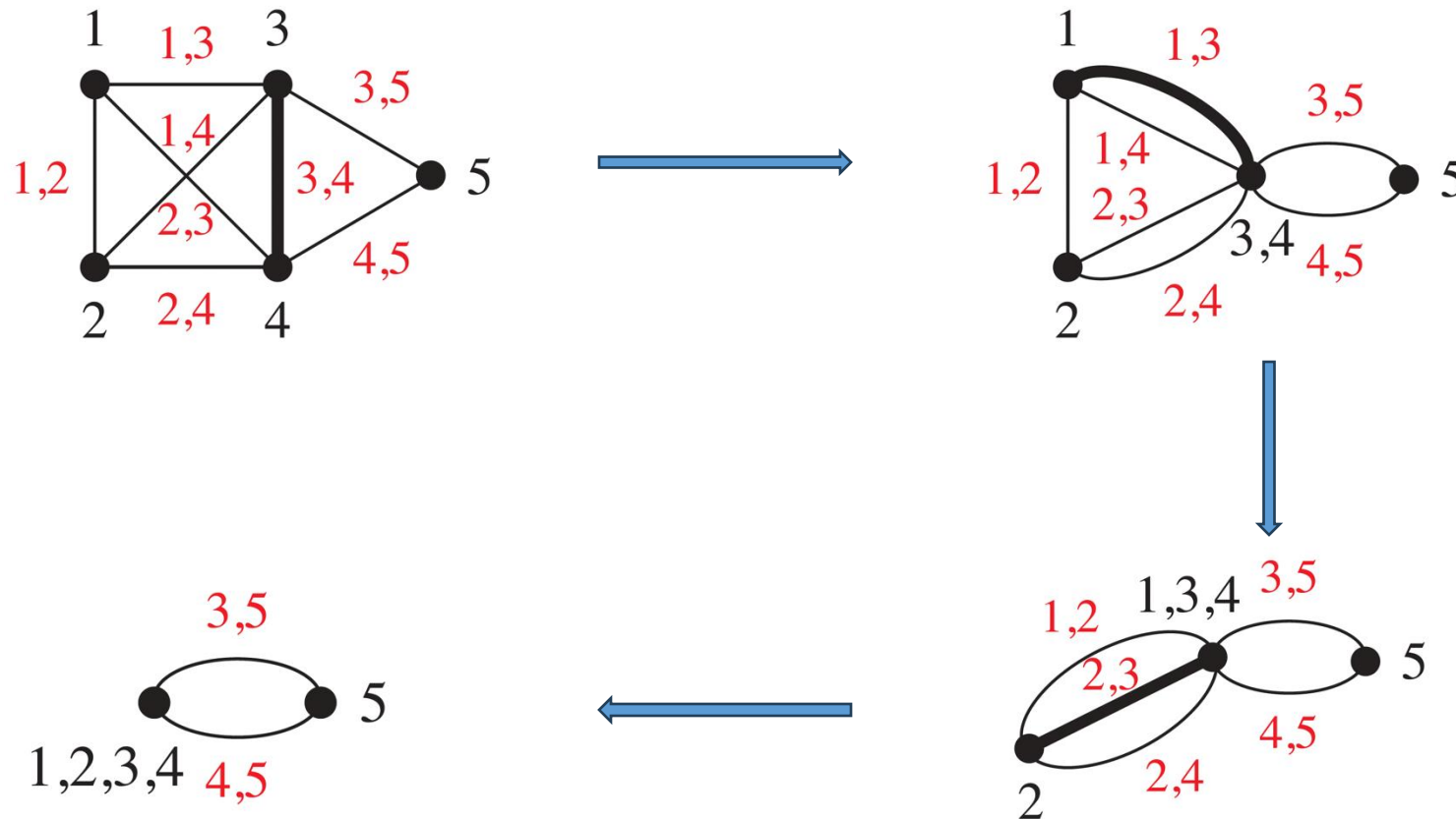
Algoritmo per il taglio minimo

Quante iterazioni sono necessarie per l'algoritmo *Randomized Min-Cut*?

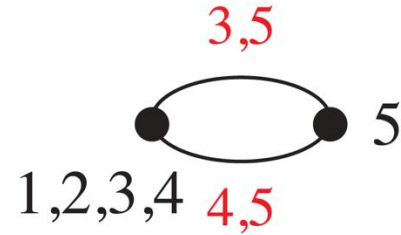
Risposta: $n - 2$.

Ad ogni iterazione togliamo esattamente un vertice, e ci fermiamo quando ne restano 2.

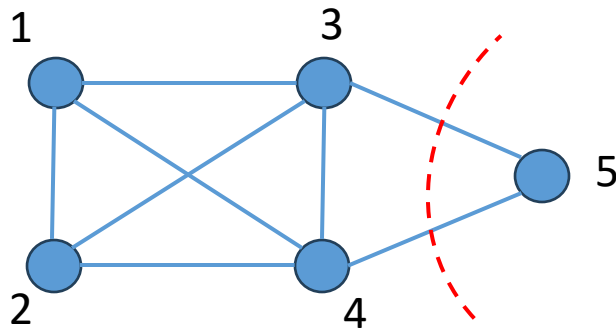
Esempio di esecuzione



Esempio di esecuzione

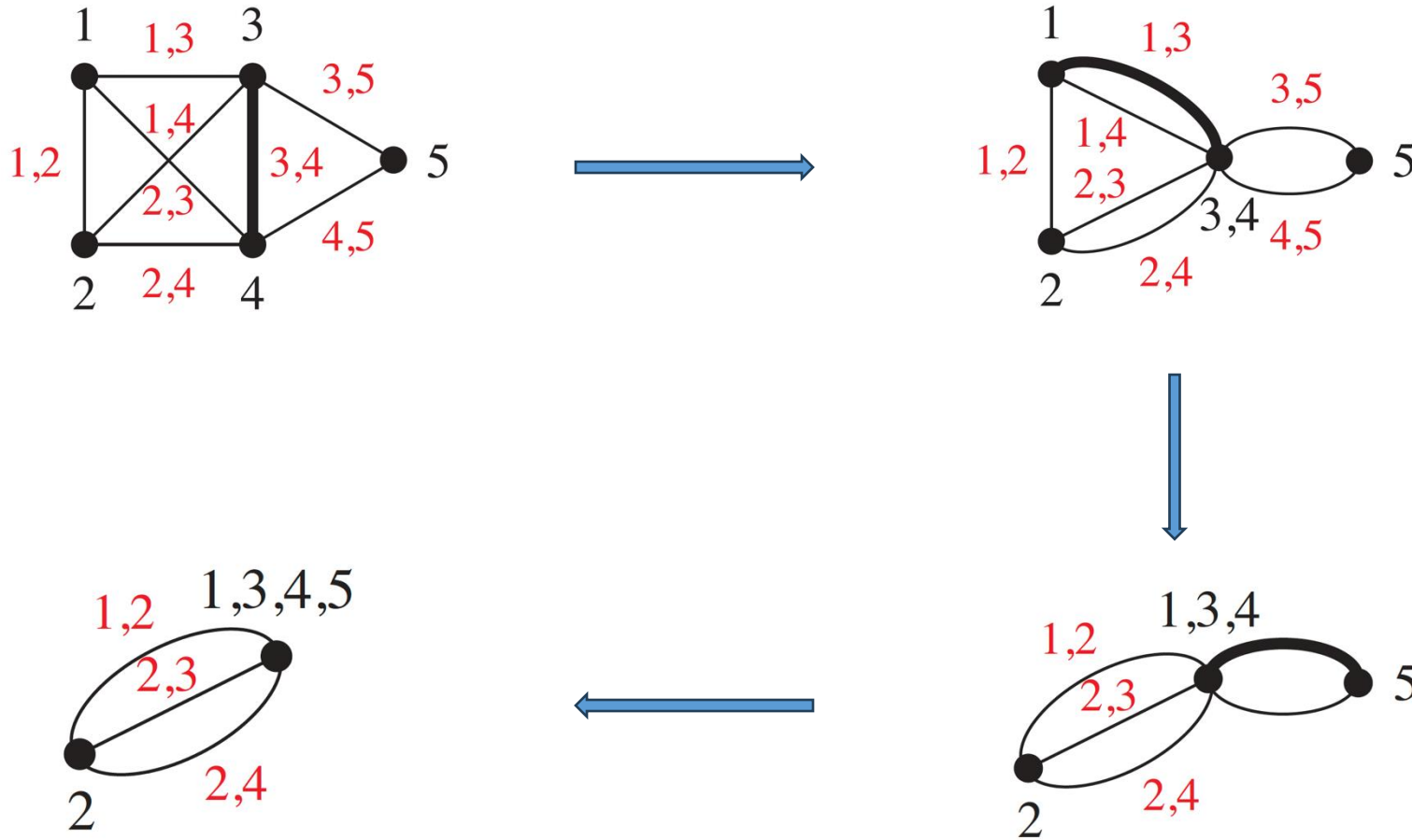


Il taglio restituito da questa esecuzione corrisponde ai **due archi** $(3,5)$ e $(4,5)$ che collegano il **vertice 5** agli altri vertici del grafo.

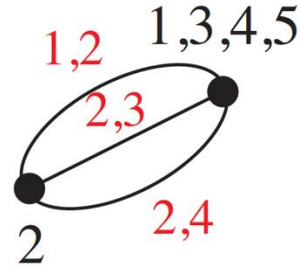


In questa esecuzione l'algoritmo ha restituito un taglio minimo: $\{(3,5), (4,5)\}$ è un **insieme di cardinalità minima** che disconnette il grafo. La risposta è **corretta**.

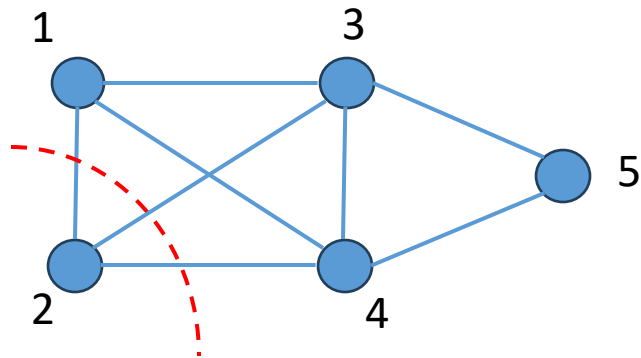
Algoritmo per il taglio minimo



Algoritmo per il taglio minimo



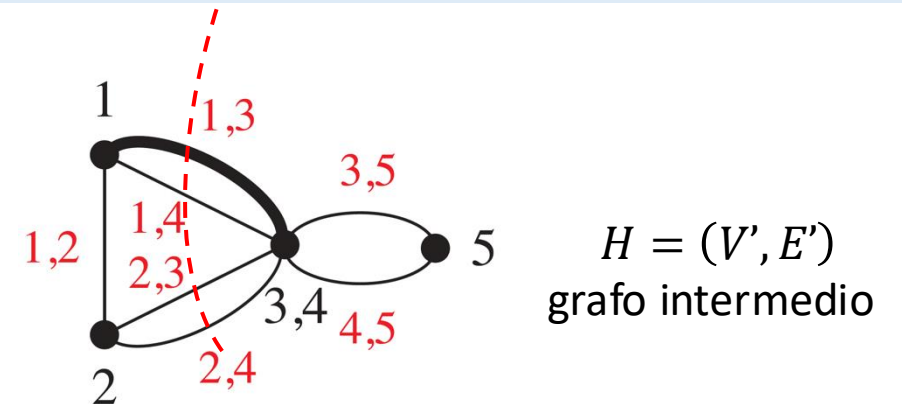
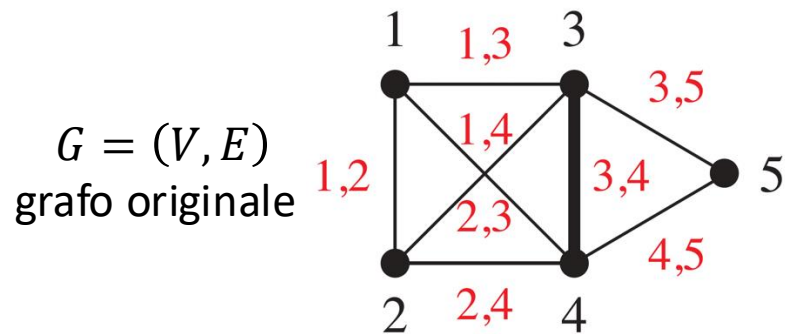
In questa esecuzione il taglio restituito corrisponde ai **tre archi** (1,2), (2,3) e (2,4) che collegano il **vertice 2** agli altri vertici del grafo.



In questa esecuzione l'algoritmo ha restituito un taglio che non è minimo. $\{(1,2), (2,3), (2,4)\}$ **non è un insieme di cardinalità minima** che disconnette il grafo. La risposta **non è corretta**.

Algoritmo per il taglio minimo

Osservazione. Sia $G = (V, E)$ il grafo originale. Qualsiasi insieme di taglio di un grafo intermedio prodotto dall'algoritmo è anche un insieme di taglio di G . Dimostrare per esercizio.



Quindi, l'output dell'algoritmo è sempre un **vero insieme di taglio** del grafo originale, ma **non necessariamente** un insieme di taglio di **dimensione minima**.

Algoritmo per il taglio minimo

Lemma 22.10

Sia C un insieme di taglio del grafo $G = (V, E)$, e sia k la **cardinalità** di C . Supponiamo di eseguire l'algoritmo *Randomized Min-Cut* per tutte le sue $n - 2$ iterazioni. Allora:

C è l'insieme di taglio restituito $\iff$ nessuno dei k archi di C viene contratto.

($\Rightarrow$) Abbiamo un taglio C (di cardinalità k) del grafo originale. Assumiamo che C sia anche il taglio restituito dall'algoritmo. Allora, ovviamente, nessuno dei k archi di C è stato contratto, altrimenti non sarebbe incluso nel taglio restituito dall'algoritmo.

Algoritmo per il taglio minimo

Lemma 22.10

Sia C un insieme di taglio del grafo $G = (V, E)$, e sia k la **cardinalità** di C . Supponiamo di eseguire l'algoritmo *Randomized Min-Cut* per tutte le sue $n - 2$ iterazioni. Allora:

C è l'insieme di taglio restituito $\iff$ nessuno dei k archi di C viene contratto.

($\Leftarrow$) Questa direzione è meno immediata. Abbiamo un taglio C (di cardinalità k) del grafo originale. Assumiamo che nessuno dei k archi di C sia stato contratto durante l'esecuzione dell'algoritmo. Dobbiamo far vedere che l'algoritmo restituisce esattamente C .

Algoritmo per il taglio minimo

Ipotesi:

1. Taglio C (di cardinalità k) del grafo originale.
2. Nessuno dei k archi di C è stato contratto durante l'esecuzione dell'algoritmo.

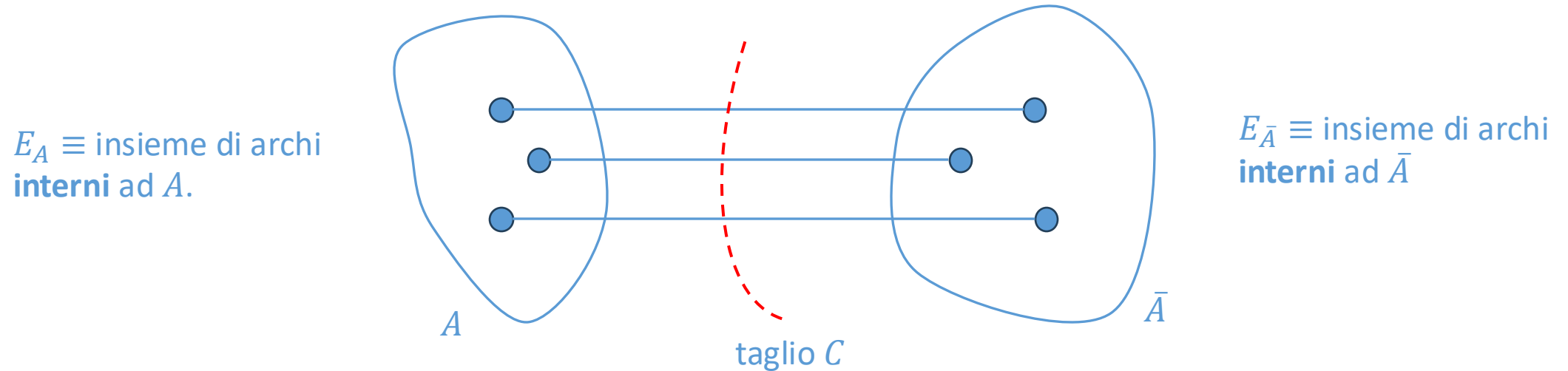
Dobbiamo far vedere che l'algoritmo restituisce esattamente C , cioè:

- Tutti gli archi di C sono inclusi nel taglio di output.
- Tutti gli archi del taglio di output sono inclusi in C .

Algoritmo per il taglio minimo

Dimostriamo che tutti gli archi di C sono inclusi nel taglio di output.

C divide il grafo in due componenti, chiamiamole A e $\bar{A}$.

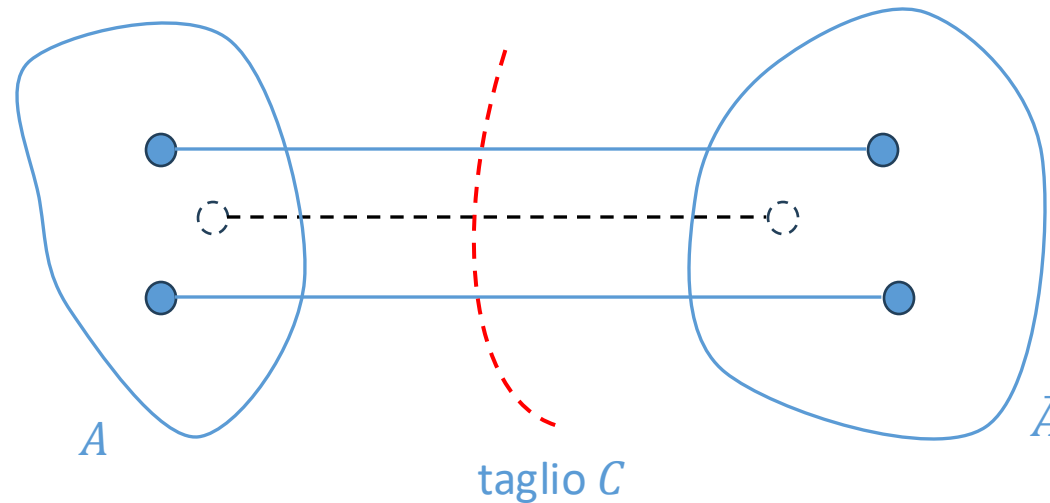


Se E_A e $E_{\bar{A}}$ indicano gli insiemi di archi **interni** ad A e a $\bar{A}$, rispettivamente, allora

$$E = C \cup E_A \cup E_{\bar{A}}.$$

Algoritmo per il taglio minimo

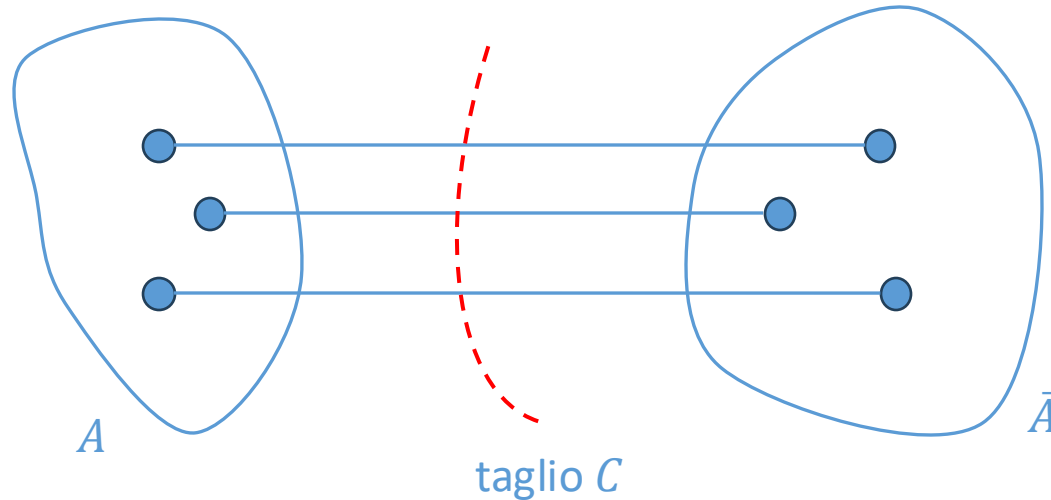
Supponiamo per assurdo che l'algoritmo non includa nel taglio restituito un arco (x, y) presente in C . Allora vuol dire che (x, y) è stato rimosso a seguito di una contrazione.



Per ipotesi, nessun arco di C è stato contratto durante l'esecuzione dell'algoritmo. Quindi, tutte le contrazioni sono avvenute per archi con entrambi gli estremi in A o in $\bar{A}$. Ma tali contrazioni non possono mai produrre la rimozione di un arco in C .

Algoritmo per il taglio minimo

Dimostriamo che tutti gli archi nel taglio di output sono in C .



Supponiamo per assurdo che l'algoritmo includa nel taglio restituito, oltre a tutti gli archi di C , anche un arco (x, y) non presente in C . Siccome C già separa G nelle componenti A e $\bar{A}$, l'arco (x, y) dovrebbe avere necessariamente un nodo in A e l'altro in $\bar{A}$. Ma se vi fosse un tale arco, allora C non sarebbe un taglio valido.